

LẬP TRÌNH WEB NÂNG CAO

Thái Thanh Tùng
FITHOU

BÀI 4: Model trong ASP.NET core MVC

- ✓ Model trong MVC
- ✓ DomainModel, ViewModel
- ✓ Create Model
- ✓ Data Annotation
- ✓ Entity Framework Core

Model trong ASP.NET core MVC

4. Model

- Trong ứng dụng Model-View-Controller (MVC), Model là dữ liệu của ứng dụng, là các lớp dữ liệu hoặc lớp nghiệp vụ logic, chịu trách nhiệm về trạng thái của ứng dụng và nghiệp vụ logic không phải giao diện người dùng.
- Logic nghiệp vụ thường được gói gọn trong model, cùng với bất kỳ logic triển khai nào để duy trì trạng thái của ứng dụng.
- Model bao gồm các loại lớp dữ liệu khác nhau với các trách nhiệm khác nhau. Một số ứng dụng MVC sẽ bao gồm một số kết hợp của các loại đối tượng Model.
- Trong các ứng dụng đơn giản, có thể chỉ sử dụng một loại lớp Model để kiểm soát trạng thái. Tuy nhiên, thường thì cách tiếp cận một loại Model này không mở rộng được cho các ứng dụng phức tạp.

Chú ý: Nghiệp vụ logic hiển thị thường không sử dụng Model mà thực hiện trong View

Model trong ASP.NET core MVC

4.1. Phân loại model

Model được phân chia theo đặc tính, 3 loại thường sử dụng gồm:

- **Domain model (Entity model):** đại diện cho lớp dữ liệu trích xuất từ cơ sở dữ liệu nguồn. Domain model thường có mối quan hệ ánh xạ 1-1 với các bảng trong CSDL. Model loại này thường liên quan đến lớp truy cập dữ liệu trong ứng dụng (DAL: Data Access Layer). Chúng được truy xuất từ cơ sở dữ liệu hoặc được lưu vào cơ sở dữ liệu bởi lớp DAL.
- **ViewModel (Data Transfer Object -DTo):** tham chiếu đến các đối tượng chứa dữ liệu cần được hiển thị cho người dùng. ViewModel liên quan đến lớp hiển thị của ứng dụng (PL: Presentation Layer). Dữ liệu được xác định dựa trên cách chúng được hiển thị cho người dùng thay vì cách được lưu trữ. Các loại ViewModel thường chỉ là container chứa dữ liệu; những nghiệp vụ logic trong loại Model này là để thực hiện chế độ View.
- **BusinessLogicModel:** Các lớp dữ liệu chứa nghiệp vụ logic, xử lý phép toán, hoặc các quy tắc xử lý dữ liệu. Những lớp dữ liệu này thực hiện cơ bản các chức năng nghiệp vụ của chương trình trước và được sử dụng bởi DomainModel hoặc ViewModel để truyền dữ liệu tới các lớp trình diễn hoặc, dữ liệu nguồn.

Ngoài ra còn các loại như API Model, Persistence Model: Tương tự như Domain, thường được liên kết với dữ liệu nguồn và được tạo bởi các tools/framework tự động. Ví dụ: EntityFrame work sẽ map Domain Model và khởi tạo Model với các phụ thuộc và tham chiếu riêng.

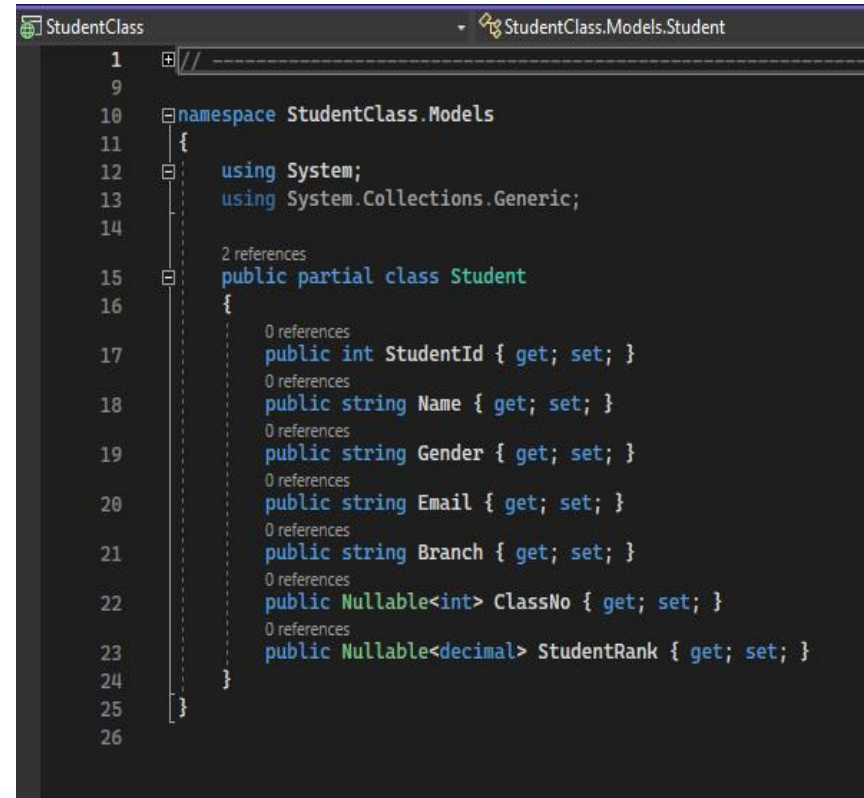
Cấu trúc Model

4.1. Phân loại Model

Ví dụ: Hệ thống quản lý sinh viên sử dụng Student là thực thể Entity dữ liệu chính.

Để thực hiện ứng dụng ta sẽ phải thiết kế Model Student như trong 4.1a và những dữ liệu này sẽ được lưu trữ xử lý phần **back-end**. Trong mô hình MVC thường gọi là Model hoặc **Domain Model**

- Ứng dụng sẽ cho phép Thêm, Sửa, Xóa, và hiển thị dữ liệu (CRUD Application)
- Nếu sinh viên có điểm trung bình thấp sẽ hiển thị cảnh báo ĐỎ, điểm cao hiển thị thông báo XANH



```
1 // -----
9
10 namespace StudentClass.Models
11 {
12     using System;
13     using System.Collections.Generic;
14
15     2 references
16     public partial class Student
17     {
18         0 references
19         public int StudentId { get; set; }
20         0 references
21         public string Name { get; set; }
22         0 references
23         public string Gender { get; set; }
24         0 references
25         public string Email { get; set; }
26         0 references
27         public string Branch { get; set; }
28         0 references
29         public Nullable<int> ClassNo { get; set; }
30         0 references
31         public Nullable<decimal> StudentRank { get; set; }
32     }
33 }
```

Hình 4.1a. DomainModel Student

Cấu trúc Model

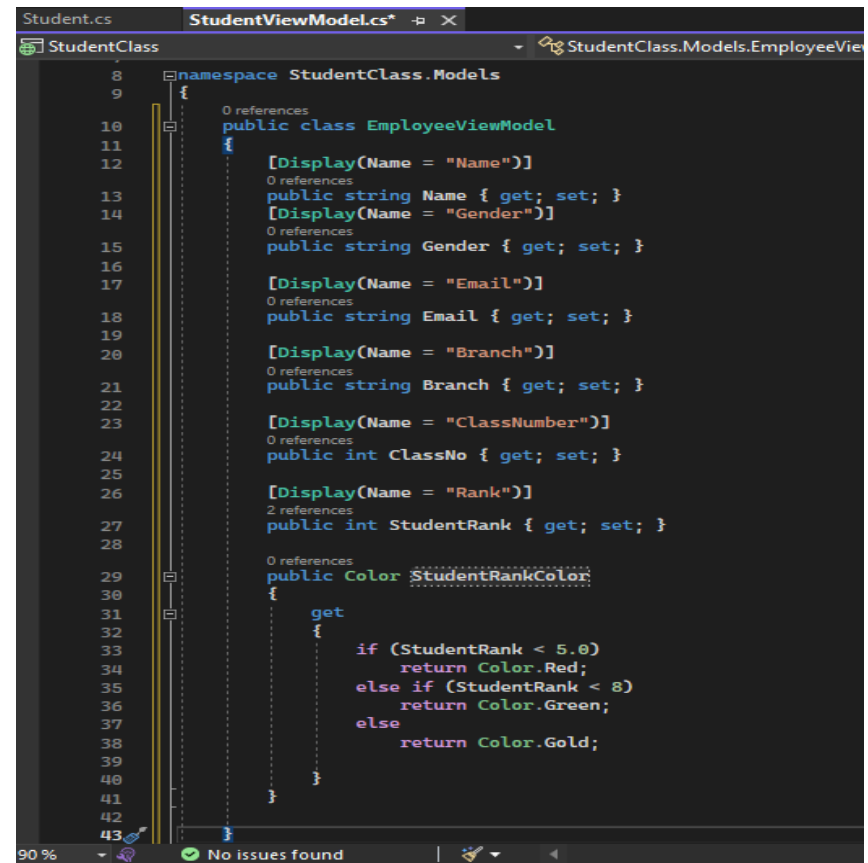
4.1. Phân loại Model

Ví dụ: Hệ thống quản lý sinh viên sử dụng Student là thực thể Entity dữ liệu chính.

Trong các chức năng CRUD thao tác trên dữ liệu đó theo những dạng thức khác nhau như Xem, Sửa ... để đảm bảo tính toàn vẹn và các ràng buộc liên kết, Ta cần thiết kế các Model để hiển thị với các View khác nhau và gọi là các **ViewModel**/Input-Edit Model

- Để hiển thị dữ liệu ta sẽ tạo ViewModel thường không bao gồm các phương thức, chức năng để đảm bảo tính an toàn

Ví dụ ViewModel với nghiệp vụ hiển thị thực thể Student được tạo như hình 4.1b



Hình 4.1b. ViewModel Student

Cấu trúc Model

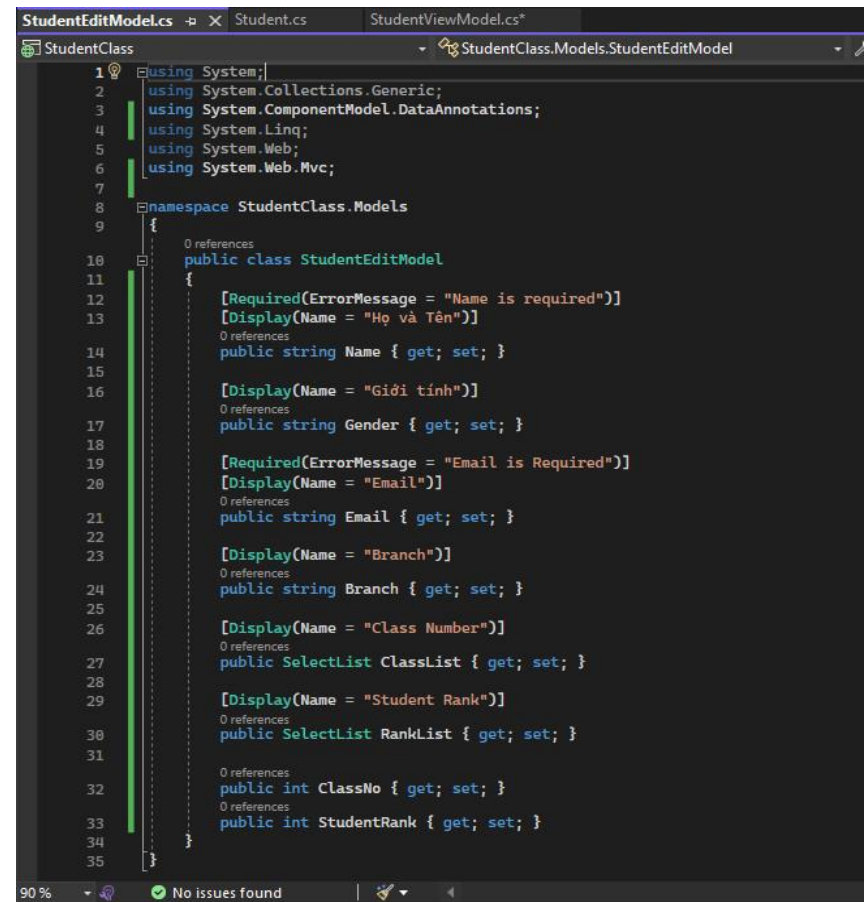
4.1. Phân loại Model

Ví dụ: Hệ thống quản lý sinh viên sử dụng Student là thực thể Entity dữ liệu chính.

Để cung cấp các chức năng thao tác trên thực thể dữ liệu, ta cần thiết kế các Model để người dùng thao tác với các View khác nhau và gọi là các **Input/Edit Model**

- Với các **chức năng** như thêm, sửa, xóa dữ liệu ta có thể tạo Edit/Input Model như hình 4.1c

Chú ý: Nếu thêm thuộc tính trên Domain để tạo ViewModel/InputModel sẽ khó xử lý trong những trường hợp nhiều layer phụ thuộc và vi phạm nguyên tắc SRP (Single responsibility principle) trong mô hình lập trình OOP.



```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel.DataAnnotations;
4 using System.Linq;
5 using System.Web;
6 using System.Web.Mvc;
7
8 namespace StudentClass.Models
9 {
10     0 references
11     public class StudentEditModel
12     {
13         [Required(ErrorMessage = "Name is required")]
14         [Display(Name = "Họ và Tên")]
15         0 references
16         public string Name { get; set; }
17
18         [Display(Name = "Giới tính")]
19         0 references
20         public string Gender { get; set; }
21
22         [Required(ErrorMessage = "Email is Required")]
23         [Display(Name = "Email")]
24         0 references
25         public string Email { get; set; }
26
27         [Display(Name = "Branch")]
28         0 references
29         public string Branch { get; set; }
30
31         [Display(Name = "Class Number")]
32         0 references
33         public SelectList ClassList { get; set; }
34
35         [Display(Name = "Student Rank")]
36         0 references
37         public SelectList RankList { get; set; }
38
39         0 references
40         public int ClassNo { get; set; }
41
42         0 references
43         public int StudentRank { get; set; }
44     }
45 }
```

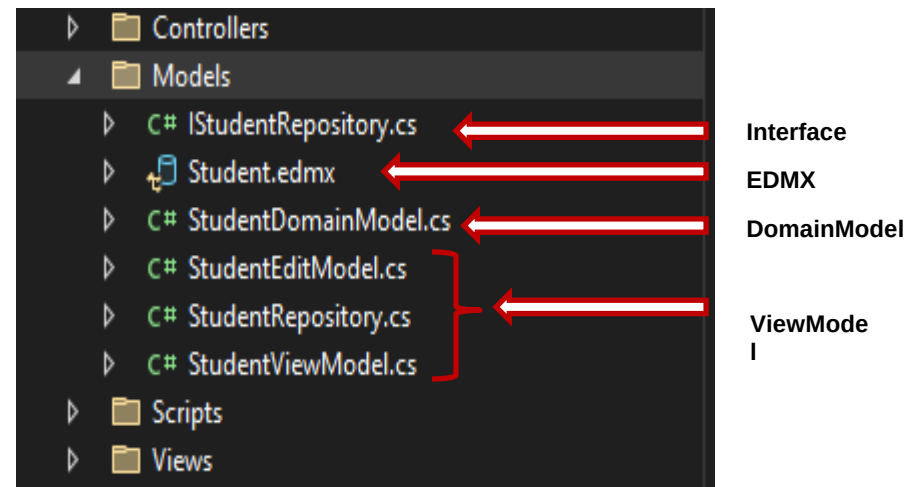
Hình 4.1c. Edit/InputModel

Cấu trúc Model

4.2. Cấu trúc Thư mục Model

Thư mục Models là nơi chứa các tệp dữ liệu Domain Model, ViewModel, InputModel. Các tệp trong thư mục Model là những tệp chương trình (*.cs/*.vb) chứa các lớp dữ liệu, thường là:

- Domain Model: Lớp Class và lớp Interface. Xử lý nghiệp vụ Business Logic Layer chương trình và Data Layer kết nối với dữ liệu từ nguồn CSDL
- ViewModel, InputModel: các lớp class với chức năng lớp Presentation Layer được gọi bởi Controller và hiển thị qua View
- Chứa các lớp dữ liệu tự động (*.edmx-ASP.NET framework) khi liên kết với dữ liệu nguồn qua Entity Framework



Hình 4.2a. Thư mục Model

Cấu trúc Model

4.2. Cấu trúc Thư mục Model

Ví dụ: Tạo Model

- StudentDomainModel.cs
Định nghĩa cấu trúc dữ liệu
- IStudentRepository.cs
Định nghĩa giao diện dữ liệu
- StudentRepository.cs
Khởi tạo kho dữ liệu

```
StudentRepository.cs | IStudentRepository.cs | StudentDomainModel.cs
StudentClass
  namespace StudentClass.Models
  {
    10 references
    public class StudentDomainModel
    {
      6 references
      public int StudentId { get; set; }
      5 references
      public string Name { get; set; }
      5 references
      public string Gender { get; set; }
      5 references
      public string Email { get; set; }
      5 references
      public string Branch { get; set; }
      5 references
      public int ClassNo { get; set; }
      5 references
      public decimal StudentRank { get; set; }
    }
  }

StudentClass
  namespace StudentClass.Models
  {
    1 reference
    public interface IStudentRepository
    {
      1 reference
      StudentDomainModel GetStudentById(int StudentId);
    }
  }
```

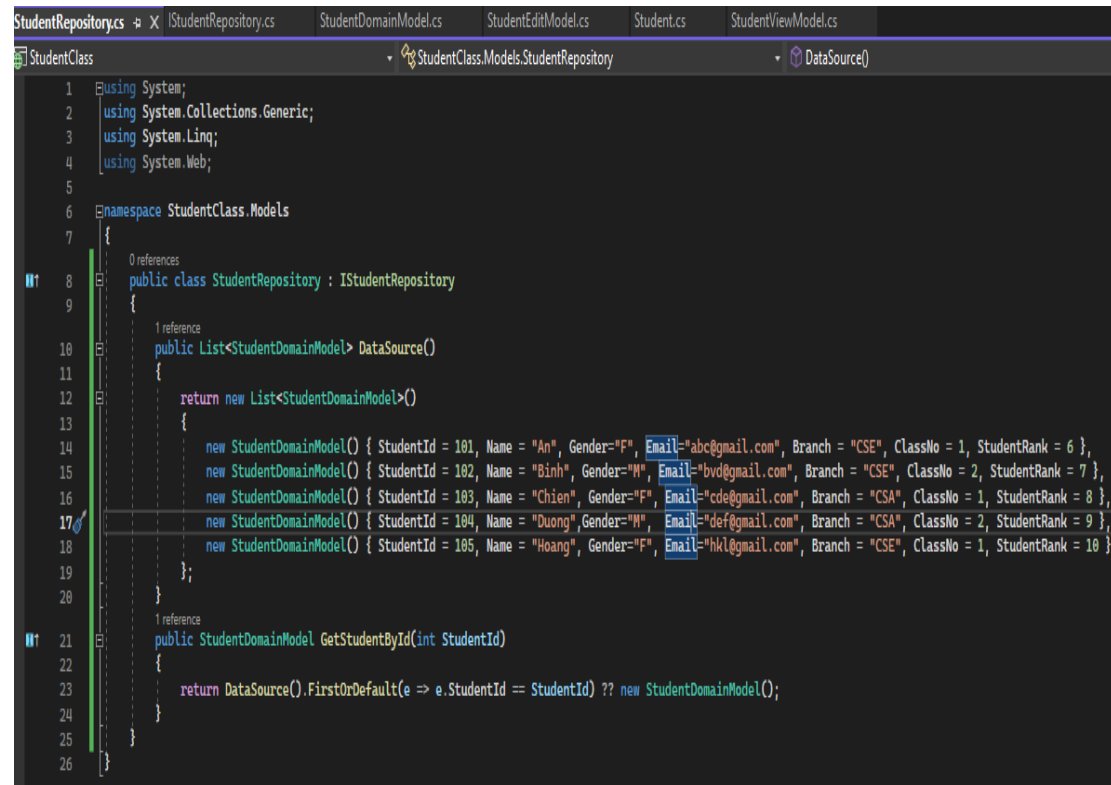
Tạo Model

Ví dụ: Tạo Model

- StudentRepository.cs
Khởi tạo dữ liệu hardcoded

Dữ liệu có thể được liên kết và truy xuất bằng sinh viên trong cơ sở dữ liệu, gọi là tiếp cận Database trước (**Database-First approach**)

Trong phần này, ta dùng hardcoded trước để tạo dữ liệu, gọi là tiếp cận (**Code-First approach**)



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5
6 namespace StudentClass.Models
7 {
8     public class StudentRepository : IStudentRepository
9     {
10         public List<StudentDomainModel> DataSource()
11         {
12             return new List<StudentDomainModel>()
13             {
14                 new StudentDomainModel() { StudentId = 101, Name = "An", Gender="F", Email="abc@gmail.com", Branch = "CSE", ClassNo = 1, StudentRank = 6 },
15                 new StudentDomainModel() { StudentId = 102, Name = "Binh", Gender="M", Email="bvd@gmail.com", Branch = "CSE", ClassNo = 2, StudentRank = 7 },
16                 new StudentDomainModel() { StudentId = 103, Name = "Chien", Gender="F", Email="cde@gmail.com", Branch = "CSA", ClassNo = 1, StudentRank = 8 },
17                 new StudentDomainModel() { StudentId = 104, Name = "Duong", Gender="M", Email="def@gmail.com", Branch = "CSA", ClassNo = 2, StudentRank = 9 },
18                 new StudentDomainModel() { StudentId = 105, Name = "Hoang", Gender="F", Email="hkl@gmail.com", Branch = "CSE", ClassNo = 1, StudentRank = 10 }
19             };
20         }
21
22         public StudentDomainModel GetStudentById(int StudentId)
23         {
24             return DataSource().FirstOrDefault(e => e.StudentId == StudentId) ?? new StudentDomainModel();
25         }
26     }
27 }
```

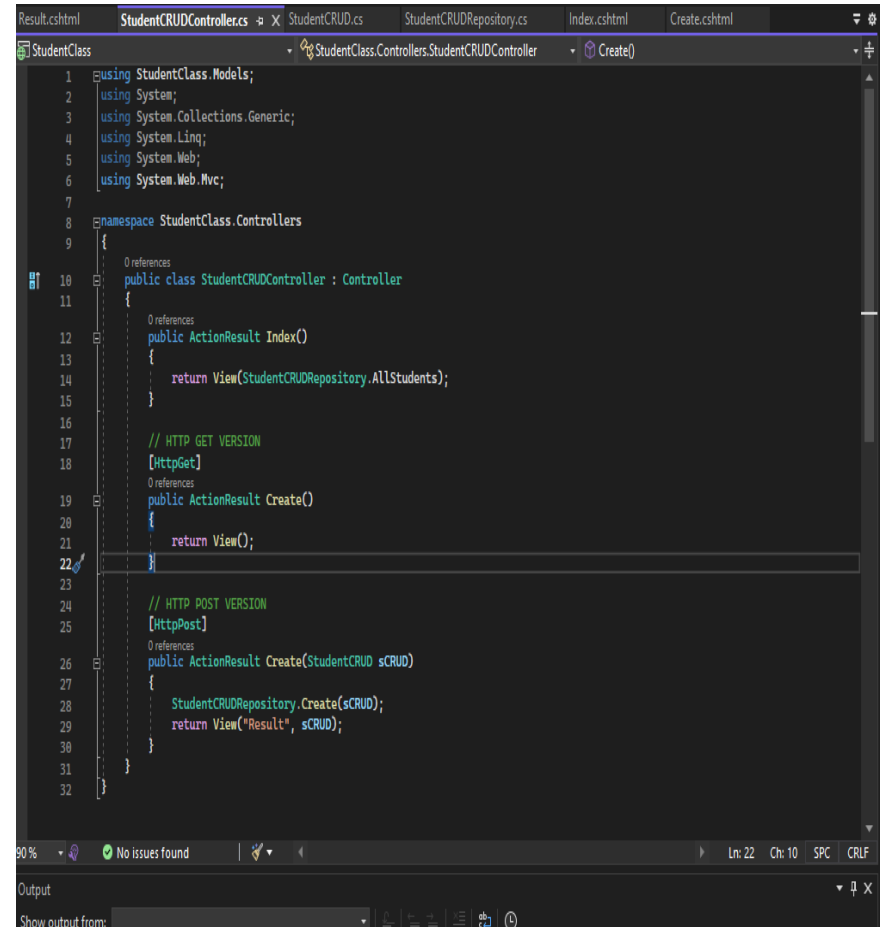
Kết hợp Model-Controller

Ví dụ: Tạo Controller gọi Model

- StudentCRUDController.cs
 - [HttpGet] trả dữ liệu về Client
 - Action method: Create()
 - [HttpPost] gửi dữ liệu lên Server
 - Action method: Create(student)

Ví dụ này tạo 2 Action Method để hiển thị bản ghi Student được tạo cho phía Client (HttpGet), và gửi lên Server để lưu trữ trong Domain Model StudentCRUDRepository

Tương tự thực hiện với các chức năng Edit/Update, Delete



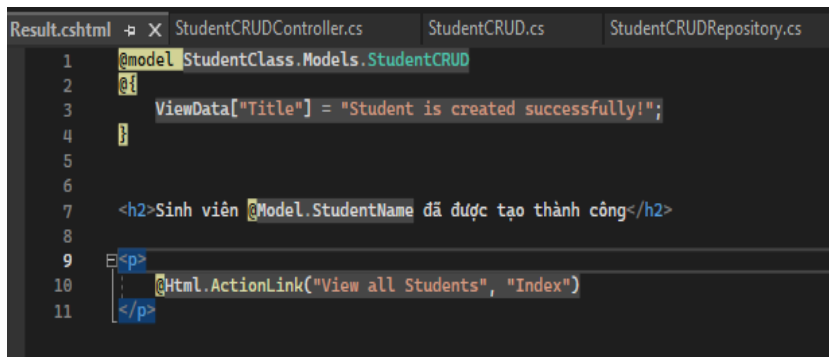
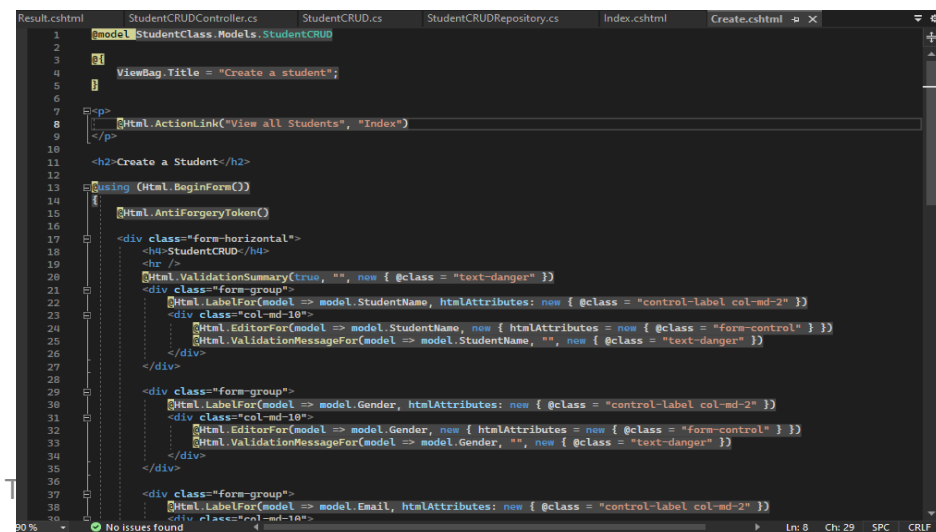
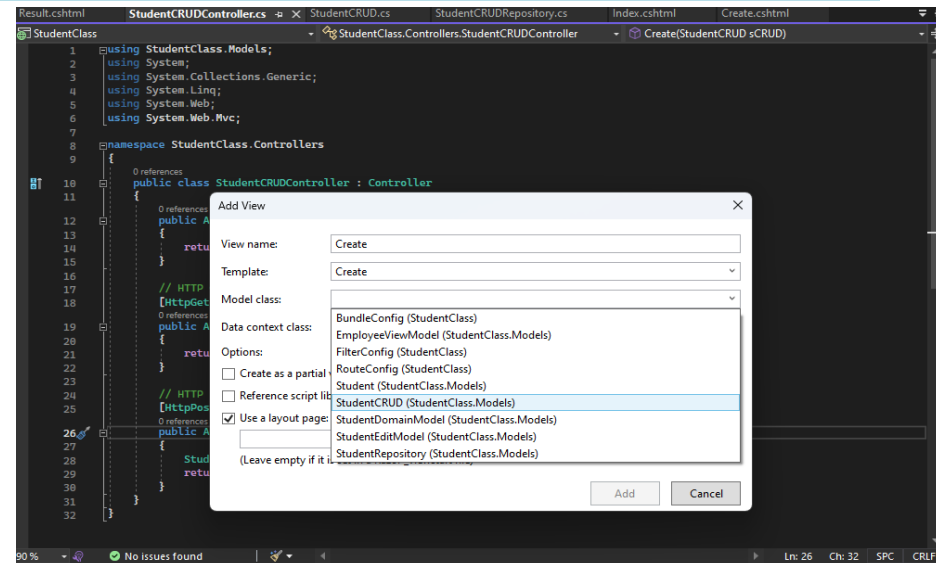
```
1 using StudentClass.Models;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Web;
6 using System.Web.Mvc;
7
8 namespace StudentClass.Controllers
9 {
10     0 references
11     public class StudentCRUDController : Controller
12     {
13         0 references
14         public ActionResult Index()
15         {
16             return View(StudentCRUDRepository.AllStudents);
17         }
18
19         // HTTP GET VERSION
20         [HttpGet]
21         0 references
22         public ActionResult Create()
23         {
24             return View();
25         }
26
27         // HTTP POST VERSION
28         [HttpPost]
29         0 references
30         public ActionResult Create(StudentCRUD sCRUD)
31         {
32             StudentCRUDRepository.Create(sCRUD);
33             return View("Result", sCRUD);
34         }
35     }
36 }
```

Kết hợp Model-View

Ví dụ: Tạo ViewModel

- Create.cshtml (HttpGet)
 - Click chuột phải chọn Add View
 - Chọn Create, Domain Model
- Result.cshtml (HttpPost)
 - Thông báo kết quả

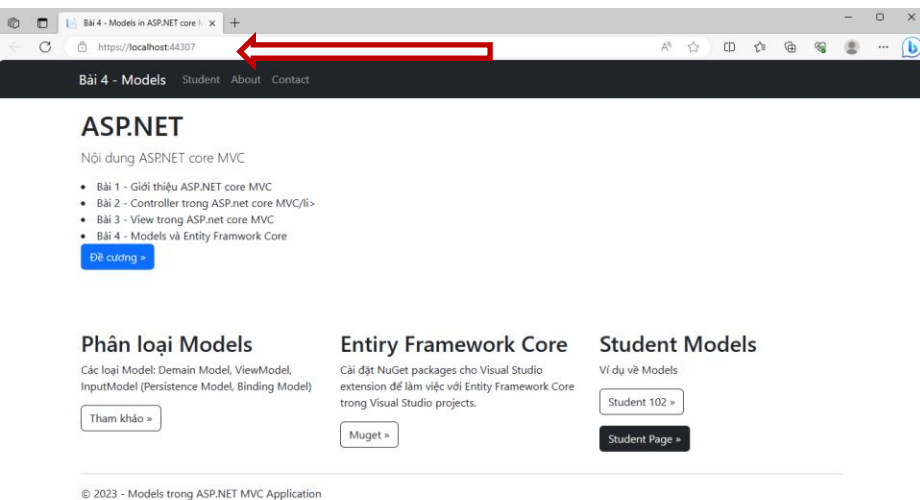
Chú ý: Thường sử dụng **HtmlHelper**, hoặc **TagHelper** để tạo các form elements cho ViewModel



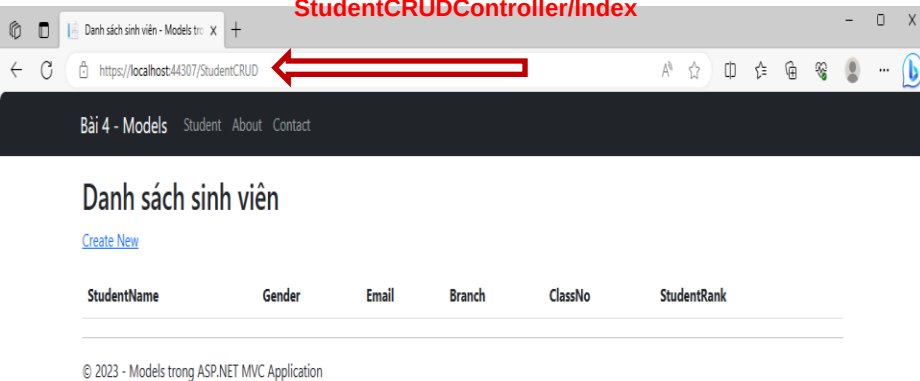
Model-View-Controller

Ví dụ: Kết quả

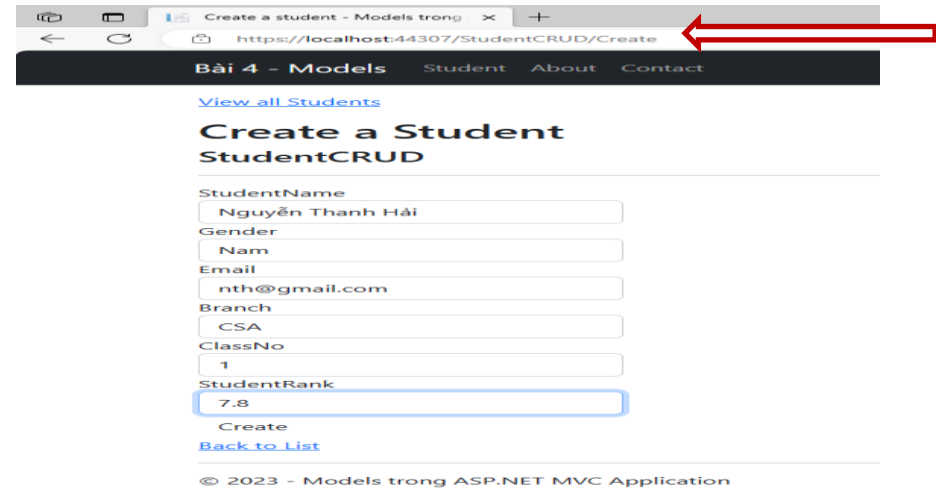
HomeController/Index



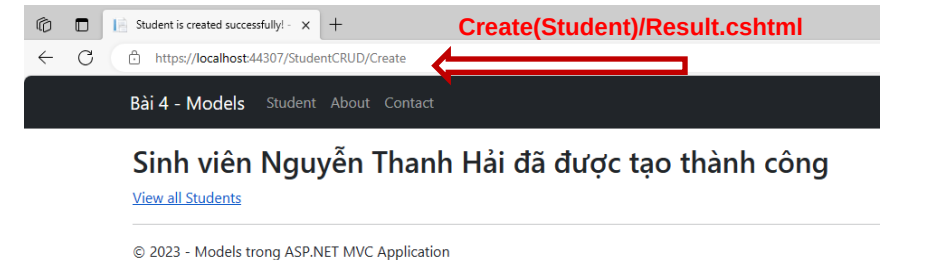
StudentCRUDController/Index



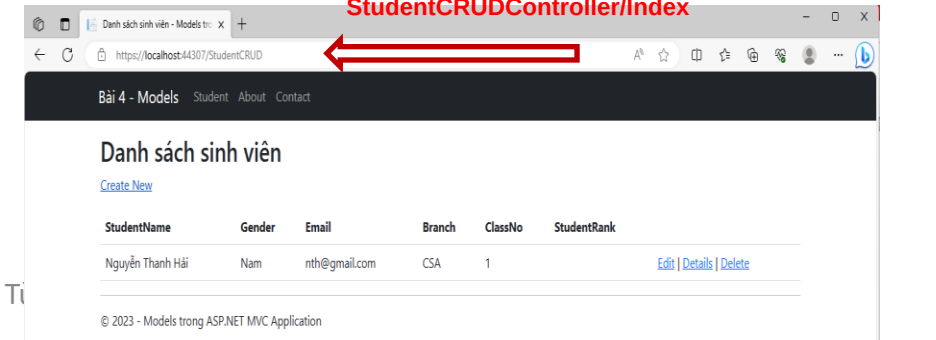
Create()/Create.cshtml



Create(Student)/Result.cshtml



StudentCRUDController/Index



Data Annotation

4.3. Data Annotation

Ngày nay, việc xác thực dữ liệu đầu vào của người dùng cho bất kỳ ứng dụng Web nào là một công việc thách thức, chiếm nhiều thời gian nhưng nhất thiết phải thực hiện. Nói một cách đơn giản, việc xác thực là những quy tắc do nhà phát triển đặt ra trên các trường dữ liệu đầu vào của trang web nhằm đáp ứng các quy tắc nghiệp vụ nhằm duy trì dữ liệu thích hợp trong hệ thống.

Web developer không chỉ xác thực logic nghiệp vụ trên trình duyệt phía Client mà còn cần xác thực logic nghiệp vụ đang chạy trên Server. Thông thường, xác thực dữ liệu phải thực hiện cả 2 loại **Validations**

- **Server-Side validations**
- **Client-Side validations**

Client-Side validations phản hồi ngay cho người dùng về dữ liệu họ đã nhập vào trang web và đây là một tính năng hỗ trợ, hướng dẫn người sử dụng nhập dữ liệu.

Server-Side validations được thực hiện để kiểm tra lại các dữ liệu được gửi lên đảm bảo cho logic nghiệp vụ của hệ thống và cơ sở dữ liệu

Data Annotation

4.3. Data Annotation

Khi xác thực dữ liệu, ta không chỉ cần quan tâm đến việc xác thực logic mà còn đảm bảo rằng việc xác thực đáp ứng quy tắc nghiệp vụ theo yêu cầu. Từ quan điểm bảo mật, cũng có thể một số tin tặc có thể disabled xác thực phía client và chèn dữ liệu dễ bị tấn công lên server.

Xác thực trong ASP.NET MVC framework, chủ yếu tập trung vào việc xác thực giá trị Model. Điều đó có nghĩa là người dùng đã cung cấp giá trị bắt buộc chưa? Giá trị có nằm trong phạm vi yêu cầu không? Giá trị có ở định dạng phù hợp không, ...?

Trong ASP.NET MVC web applications, ta có thể thực hiện 3 dạng thức validations:

- HTML validation / JavaScript validation (Client-Side Validation)
- ASP.NET MVC Model validation (Server-side Validation)
- Database validation (Server-side Validation)

ASP.NET MVC Framework cung cấp một khái niệm gọi là Chú thích dữ liệu (Data Annotation) được sử dụng để xác thực mô hình. Kế thừa từ 2 Namespaces

- **System.ComponentModel.DataAnnotations**
- **System.ComponentModel.DataAnnotations.Schema**

Data Annotation

Assembly

System.ComponentModel.DataAnnotations

- Key
- Timestamp
- ConcurrencyCheck
- Required
- MinLength
- MaxLength
- StringLength
- ScaffoldColumn
- Bind
- DisplayName
-

System.ComponentModel.DataAnnotations.Schema

- Table
- Column
- Index
- ForeignKey
- NotMapped
- InverseProperty

Data type

Sử dụng kiểu dữ liệu để xác thực dữ liệu được nhập vào Input Form Element từ phía users

- Credit Card number
- Currency
- Custom
- Date
- DateTime
- Duration
- Email Address
- HTML
- Image URL
- Multiline text
- Password
- Phone number
- Postal Code
- Upload

Data Annotation

4.3.1. Front-End Validations

ASP.NET core MVC Framework cung cấp cả xác thực phía Client và Server. Có 2 bước đơn giản để kích hoạt xác thực phía máy khách trong ứng dụng ASP.NET coreMVC.

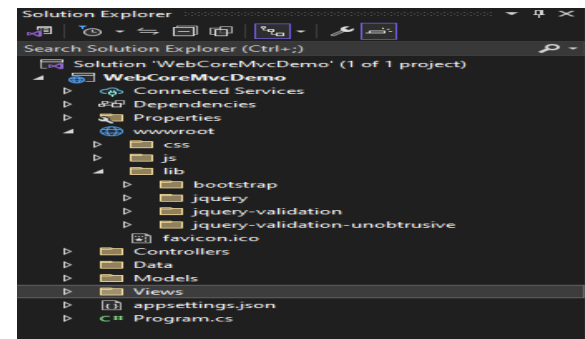
Sử dụng để thực thi xác thực các nghiệp vụ logic, các quy tắc xác thực trong chế độ hiển thị **ViewModel** và kích hoạt **Client-Side Validations**. Thường dùng để kiểm tra các dữ liệu input đầu vào như username, mật khẩu, email, dữ liệu, các trường có mặt nạ ... ngay khi thực thi phía Brower.

ASP.NET MVC sử dụng jQuery, và jQuery Validation, jQuery Unobtrusive Validation để thực hiện xác thực dữ liệu phía Client. Để sử dụng ta cần cài đặt 3 packages này qua NuGet Package Manager Console

Tools => NuGet Package Manager

Package Manager Console (ASP.NET Framework)

- PM> Install-Package jQuery
- PM> Install-Package jQuery.Validation
- PM> Install-Package jQuery.Validation.Unobtrusive



Chú ý: .Net 6.0 được tích hợp sẵn các thư viện trong wwwroot/lib/

Data Annotation

4.3.1. Front-End Validations

Kích hoạt Client-Side Validations

1. **Controller:** Thiết lập Action Verbs gửi dữ liệu lên Server (**HttpPost, HttpPut, HttpPatch ...**)
2. **View:** Gọi thư viện jquery.validate và jquery.validate.unobtrusive, sử dụng Tag.Helper để kích hoạt Client-Side Script Validation. (Package CodeGeneration.Design tự động sinh Tag Helper trên Client)

Ví dụ:

StudentController

```
StudentController.cs Student.cs WebCoreMvc

public IActionResult Create()
{
    return View();
}

// HTTP POST VERSION
[HttpPost]
public IActionResult Create(Student aStudent)
{
    try
    {
        if (ModelState.IsValid)
        {
            StudentRepository.Create(aStudent);
            return View("Result", aStudent);
        }
        return View();
    }
    catch
    {
        return View();
    }
}

[HttpGet]
public IActionResult Update(string sname)
```

_Layout.cshtml

```
<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>
@await RenderSectionAsync("Scripts", required: false)
```

Create.cshtml

```
@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

Data Annotation

4.3.1. Front-End Validations Attributes

Ví dụ: <https://localhost:xxxx/Student/Create>

```
7 <div>Tạo sinh viên mới</div>
8 <p><a asp-action="Details">Xem danh sách sinh viên</a></p>
9
10 <div class="row">
11 <div class="col-md-4">
12 <div class="form-actions">Create</div>
13 <div class="form-group">
14 <label asp-for="Name" class="control-label"></label>
15 <input asp-for="Name" class="form-control" />
16 <span asp-validation-for="Name" class="text-danger"></span>
17 </div>
18 <div class="form-group">
19 <label asp-for="Gender" class="control-label"></label>
20 <input asp-for="Gender" class="form-control" />
21 <span asp-validation-for="Gender" class="text-danger"></span>
22 </div>
23 <div class="form-group">
24 <label asp-for="DOB" class="control-label"></label>
25 <input asp-for="DOB" class="form-control" />
26 <span asp-validation-for="DOB" class="text-danger"></span>
27 </div>
28 <div class="form-group">
29 <label asp-for="Email" class="control-label"></label>
30 <input asp-for="Email" class="form-control" />
31 <span asp-validation-for="Email" class="text-danger"></span>
32 </div>
33 <div class="form-group">
34 <label asp-for="Branch" class="control-label"></label>
35 <input asp-for="Branch" class="form-control" />
36 <span asp-validation-for="Branch" class="text-danger"></span>
37 </div>
38 <div class="form-group">
39 <label asp-for="ClassNo" class="control-label"></label>
40 <input asp-for="ClassNo" class="form-control" />
41 <span asp-validation-for="ClassNo" class="text-danger"></span>
42 </div>
43 <div class="form-group">
44 <label asp-for="Grade" class="control-label"></label>
45 <input asp-for="Grade" class="form-control" />
46 <span asp-validation-for="Grade" class="text-danger"></span>
47 </div>
48 <div class="form-group">
49 <label asp-for="Rank" class="control-label"></label>
50 <input asp-for="Rank" class="form-control" />
51 <span asp-validation-for="Rank" class="text-danger"></span>
52 </div>
53 <div class="form-group">
54 <input type="submit" value="Create" class="btn btn-primary" />
55 </div>
56 </div>
57 </div>
58
59 <div>
60 <a asp-action="Details">Quay lại danh sách</a>
61 </div>
62
63
64
65 @section Scripts {
66     @await Html.RenderPartialAsync("_ValidationScriptsPartial");
67 }
68
```

Tạo mới sinh viên - WebCoreMvc

<https://localhost:7021/Student/Create>

WebCoreMvcDemo Trang chủ Sinh viên

Tạo sinh viên mới

[Xem danh sách sinh viên](#)

Student Name

The Student Name field is required.

Gender

DOB

The DOB field is required.

Email

Branch

ClassNo

The ClassNo field is required.

Grade

The Grade field is required.

Rank

The Rank field is required.

[Quay lại danh sách](#)

Data Annotation

4.3.2. Back-End Model Attributes

Sử dụng để thực thi xác thực các nghiệp vụ logic và các quy tắc xác thực trong **Model** và kích hoạt **Server-Side Validations**. Thường dùng để kiểm tra các Domain Model, ViewModel trên các trường dữ liệu thiết kế để đảm bảo tính đúng đắn của dữ liệu. Để sử dụng:

1. Khai báo Namespace DataAnnotations

System.ComponentModel.DataAnnotations

2. Định nghĩa xác thực trong Model

[Attributes(ErrorMessage)]

[DataType(DataType.type, ErrorMessage)]

[CustomValidationClass(ErrorMessage)]

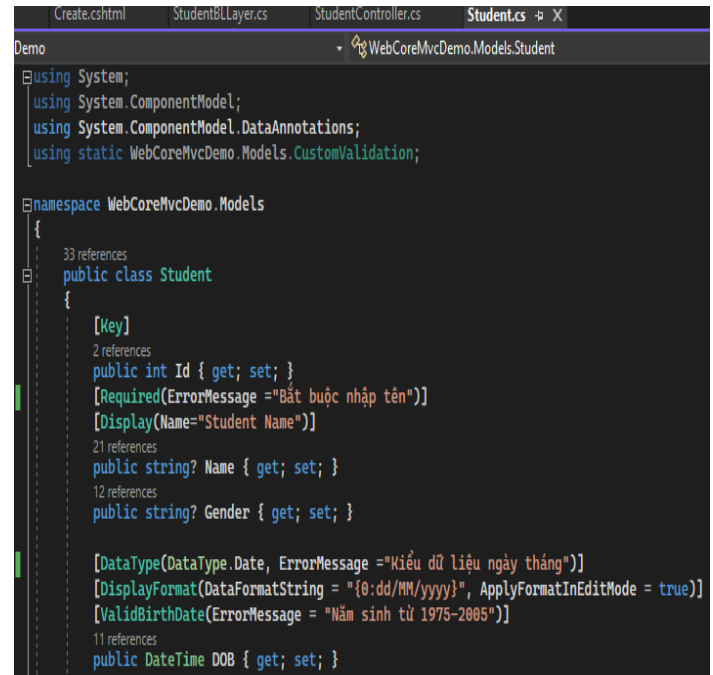
3. Điều khiển hiển thị (ví dụ trên)

ValidationMessage

ValidationMessageFor

ValidationSummary

Chú ý: RemoteValidation thường dùng Ajax Request



```
using System;
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;
using static WebCoreMvcDemo.Models.CustomValidation;

namespace WebCoreMvcDemo.Models
{
    33 references
    public class Student
    {
        [Key]
        2 references
        public int Id { get; set; }
        [Required(ErrorMessage = "Bắt buộc nhập tên")]
        [Display(Name = "Student Name")]
        21 references
        public string? Name { get; set; }
        12 references
        public string? Gender { get; set; }

        [DataType(DataType.Date, ErrorMessage = "Kiểu dữ liệu ngày tháng")]
        [DisplayFormat(DataFormatString = "{0:dd/MM/yyyy}", ApplyFormatInEditMode = true)]
        [ValidBirthDate(ErrorMessage = "Năm sinh từ 1975-2005")]
        11 references
        public DateTime DOB { get; set; }
    }
}
```

Data Annotation

4.3.2. Back-End Model Attributes

- **Custom Validation:** định nghĩa lớp CustomValidation kế thừa ValidationAttribute

```
using System.ComponentModel.DataAnnotations;

namespace WebCoreMvcDemo.Models
{
    1 reference
    public class CustomValidation
    {
    }

    1 reference
    public sealed class ValidBirthDate : ValidationAttribute
    {
        0 references
        protected override ValidationResult IsValid(object value, ValidationContext validationContext)
        {
            DateTime sDOB = Convert.ToDateTime(value);
            DateTime minDate = Convert.ToDateTime("01-1-1975");
            DateTime maxDate = Convert.ToDateTime("31-12-2005");

            if (sDOB > minDate && sDOB < maxDate)
                return ValidationResult.Success;
            else
                return new ValidationResult(ErrorMessage);
        }
    }
    1 reference
```

```
using static WebCoreMvcDemo.Models.CustomValidation;

namespace WebCoreMvcDemo.Models
{
    33 references
    public class Student
    {
        [Key]
        2 references
        public int Id { get; set; }
        [Required(ErrorMessage = "Bắt buộc nhập tên")]
        [Display(Name = "Student Name")]
        21 references
        public string? Name { get; set; }
        12 references
        public string? Gender { get; set; }

        [DataType(DataType.Date, ErrorMessage = "Kiểu dữ liệu ngày tháng")]
        [DisplayFormat(DataFormatString = "{0:dd/MM/yyyy}", ApplyFormatInEditMode = true)]
        [ValidBirthDate(ErrorMessage = "Năm sinh từ 1975-2005")]
        11 references
        public DateTime DOB { get; set; }
        12 references
    }
}
```

Data Annotation

Ví dụ: Validation Attributes

```
DataAnnotationController.cs | DataAnnotationModel.cs
StudentClass
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel.DataAnnotations;
4 using System.Linq;
5 using System.Web;
6
7 namespace StudentClass.Models
8 {
9     1 reference
10     public class DataAnnotationModel
11     {
12         [Required(ErrorMessage = "Bắt buộc nhập tên")]
13         [StringLength(30, MinimumLength = 4,
14             ErrorMessage = "Từ 4 đến 30 ký tự"),
15         [RegularExpression(@"^[A-Za-z]+\s+[A-Za-z]+$",
16             ErrorMessage = "Gồm chữ hoa chữ thường, không có dấu cách")]
17         0 references
18         public string Name
19         {
20             get;
21             set;
22         }
23     }
24 }
25
26
DataAnnotationController.cs | DataAnnotationModel.cs
StudentClass
1 using StudentClass.Models;
2 using System;
3 using System.Collections.Generic;
4 using System.ComponentModel.DataAnnotations;
5 using System.Linq;
6 using System.Web;
7 using System.Web.Mvc;
8
9 namespace StudentClass.Controllers
10 {
11     0 references
12     public class DataAnnotationController : Controller
13     {
14         // GET: DataAnnotation
15         0 references
16         public ActionResult Index()
17         {
18             return View();
19         }
20
21         [HttpPost]
22         0 references
23         public ActionResult Index(DataAnnotationModel dataAnnotation)
24         {
25             return View();
26         }
27     }
28 }
```

https://localhost:44307/DataAnnotation/Index

Bài 4 - Models Student About Contact

Index

DataAnnotationModel

Name

Bắt buộc nhập tên

Create

[Back to List](#)

© 2023 - Models trong ASP.NET MVC Application

Bài 4 - Models Student About Contact

Index

DataAnnotationModel

Name

Từ 4 đến 30 ký tự

Create

[Back to List](#)

© 2023 - Models trong ASP.NET MVC Application

Bài 4 - Models Student About Contact

Index

DataAnnotationModel

Name

Gồm chữ hoa chữ thường, không có dấu cách

Create

[Back to List](#)

© 2023 - Models trong ASP.NET MVC Application

Entity Framework Core

4.4. Entity Framework Core

- Entity Framework (EF) Core là ORM (Object-Relational Mapper) Framework để truy cập dữ liệu trong .NET Core, là phiên bản mở rộng, nhẹ, mã nguồn mở và đa nền tảng của công nghệ truy cập dữ liệu Entity Framework. Nó hoạt động trên nhiều hệ điều hành như Windows, Mac và Linux.
- EF (O/RM) để ánh xạ các đối tượng tới cơ sở dữ liệu quan hệ, là một framework cải tiến của ADO.NET, tự động hóa quá trình truy cập và lưu trữ dữ liệu trong cơ sở dữ liệu. EF Core được thiết kế sử dụng với các ứng dụng .NET tiêu chuẩn dựa trên Framework 4.5 trở lên.

Application Types	<u>ASP.NET Core Applications</u> Web, API, Console, etc.	<u>.NET 4.5+ Applications</u> Console, WinForm, WPF, ASP.NET	Devices + IoT, Mobile, PC, Xbox, Surface Hub	<u>Mobile Application</u> Android, iOS, Windows
EF Core	EF Core	EF Core	EF Core	EF Core
Framework	.NET Core	.NET 4.5+	UWP	Xamarin
OS	Windows, Mac, Linux	Windows	Windows 10	Mobile

Hình 4.4. Entity Framework Core

Entity Framework Core

4.4.1. Mô hình phát triển với EF Core (ORM)

EF Core 6.0 là phiên bản long term support hỗ trợ nhiều loại CSDL khác nhau

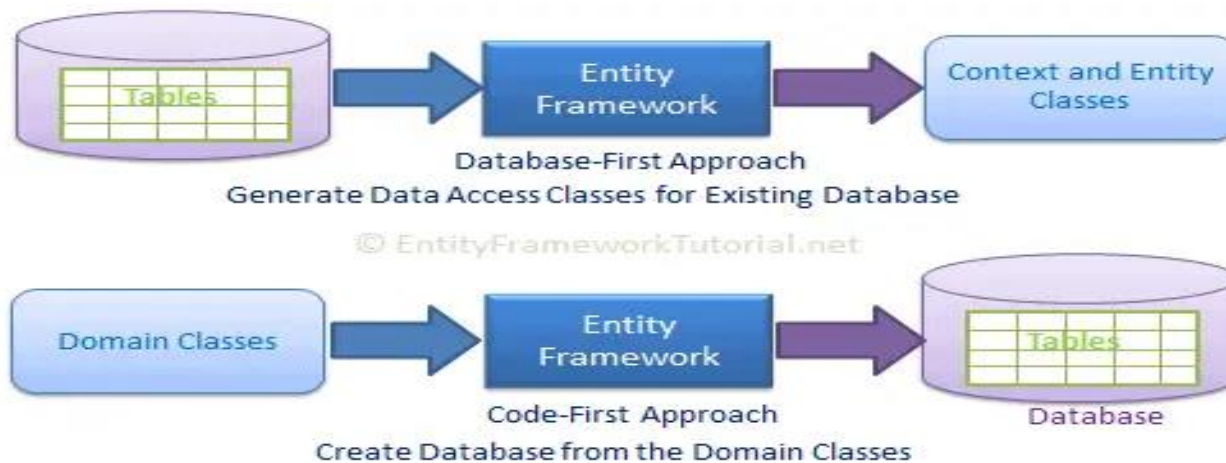
EF Core Version	Release Date	Target Framework	Database	NuGet Package
EF Core 7.0	Nov 2022	.NET 6	SQL Server	Microsoft.EntityFrameworkCore.SqlServer
EF Core 6.0	Nov 2021	.NET 6	MySQL	MySQL.Data.EntityFrameworkCore
EF Core 5.0	Nov 2020	.NET Standard 2.1	PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL
EF Core 3.0	Sept 2019	.NET Standard 2.1	SQLite	Microsoft.EntityFrameworkCore.SQLite
EF Core 2.0	August 2017	.NET Standard 2.0	SQL Compact	EntityFrameworkCore.SqlServerCompact40
EF Core 1.0	June 2016	.NET Standard 2.1	In-memory	Microsoft.EntityFrameworkCore.InMemory

Entity Framework Core version và Database Provider

Entity Framework Core

4.4.1. Mô hình phát triển với EF Core (ORM)

- Mã đầu tiên: EF Core API tạo cơ sở dữ liệu và bảng dựa trên các quy ước và cấu hình được cung cấp trong các Domain Model.
- Cơ sở dữ liệu: EF Core API tạo các Domain Model và Context dựa trên cơ sở dữ liệu có sẵn. Tiếp cận này bị hạn chế vì nó không hỗ trợ thiết kế trực quan.



Hình 4.4.1. Mô hình phát triển với Entity Framework Core

Entity Framework Core

4.4.2. Cài đặt EF Core từ NuGet

- EF Core 7 là phiên bản mới nhất của Entity Framework (EF Core 6.0 long term support):
Microsoft.EntityFrameworkCore
- Local SQL Server: **Microsoft.EntityFrameworkCore.SqlServer**

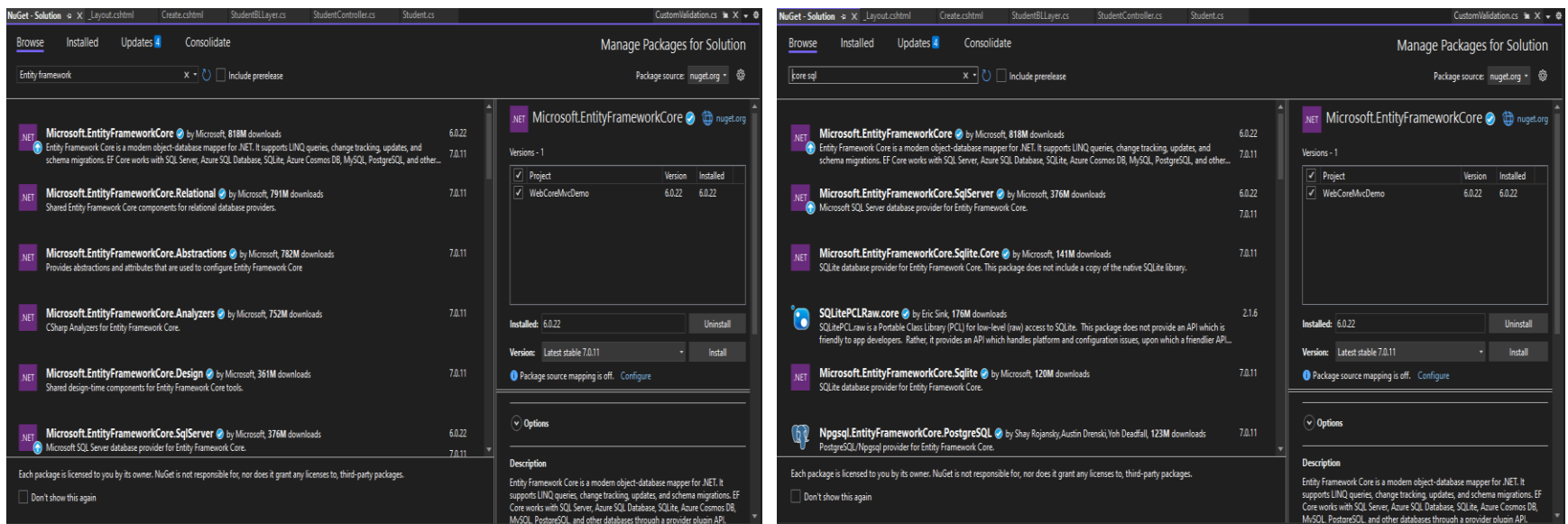
Cài đặt:

- Tools > NuGet Package Manager > Manage Package for Solution, hoặc
- Tools > NuGet Package Manager > Package Manager Console
PM> Install-Package Microsoft.EntityFrameworkCore.SqlServer
- Có thể sử dụng Command Line Interface (CLI), hoặc Power Shell

Entity Framework Core

4.4.2. Cài đặt EF Core

- EF Core 7 là phiên bản mới nhất của Entity Framework (EF Core 6.0 long term support)
- Cơ sở dữ liệu Local SQL Server sử dụng **Microsoft.EntityFrameworkCore.SqlServer** (NuGet package)

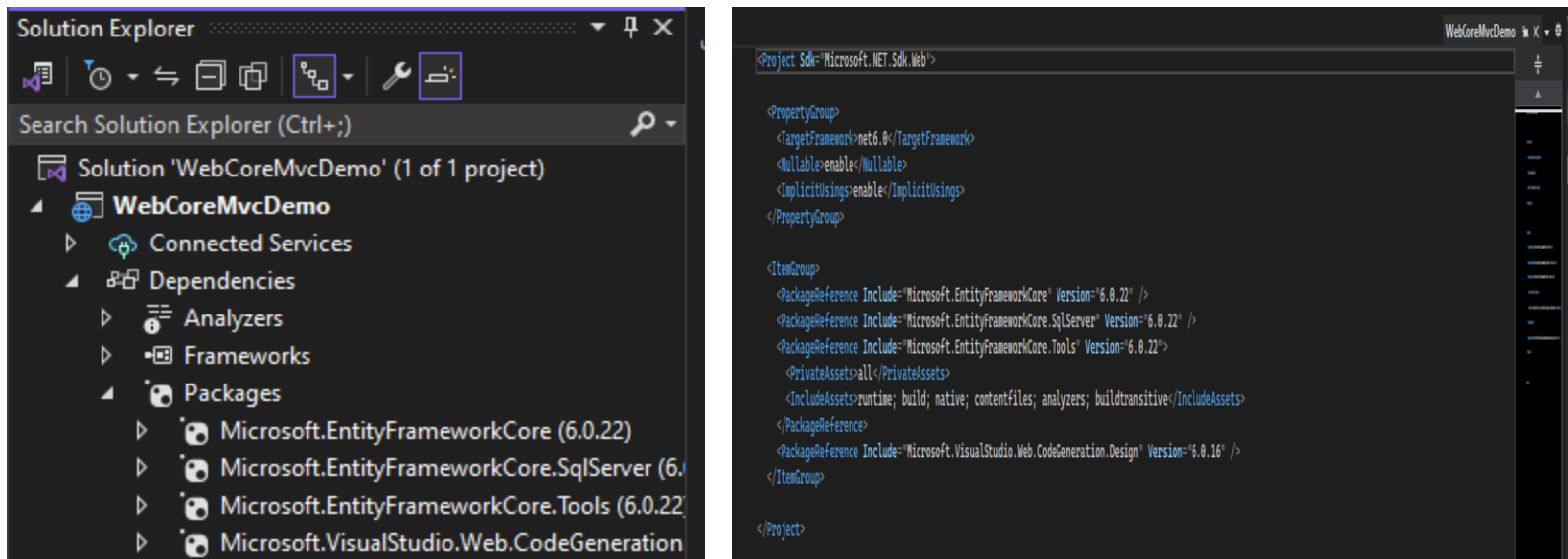


Hình 4.4.2a. Entity Framework Core Packages

Entity Framework Core

4.4.2. Cài đặt EF Core

- Packages và Project Item trong Solution



Hình 4.4.2b. Entity Framework Core Packages

Entity Framework Core

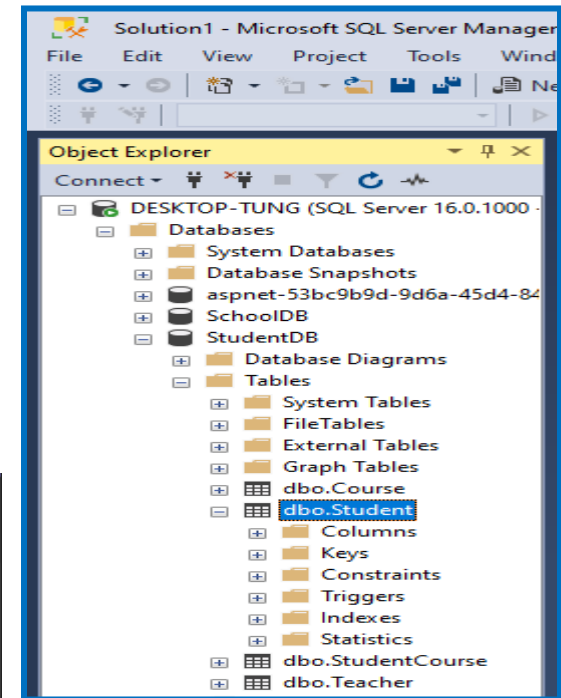
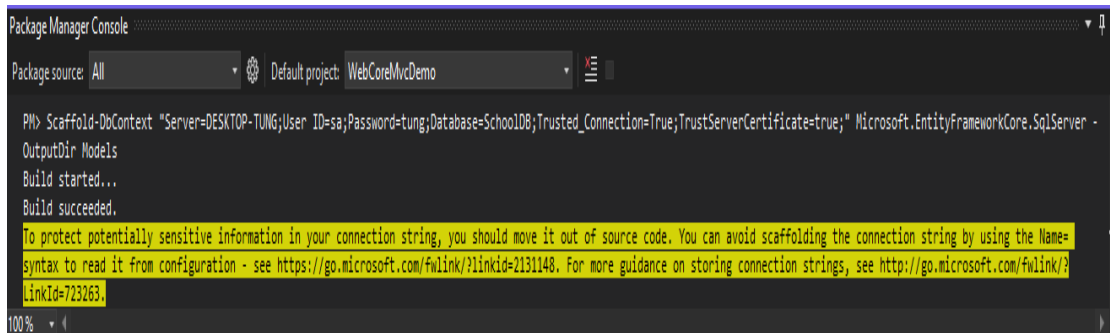
4.4.3. Truy xuất dữ liệu với EF Core

DB-First: Scaffold-DbContext Command

- Tools > NuGet Package Manger > Package Manger Console
- **Scaffold-DbContext** [-Connection] [-Provider] [-OutputDir] [-Context] [-Schemas>] [-Tables>] [-DataAnnotations] [-Force] [-Project] [-StartupProject] [<CommonParameters>]

Ví dụ:

Scaffold-DbContext "Server=DESKTOP-TUNG;User ID=sa;Password=tung;Database=SchoolDB;Trusted_Connection=True;TrustServerCertificate=true;" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models

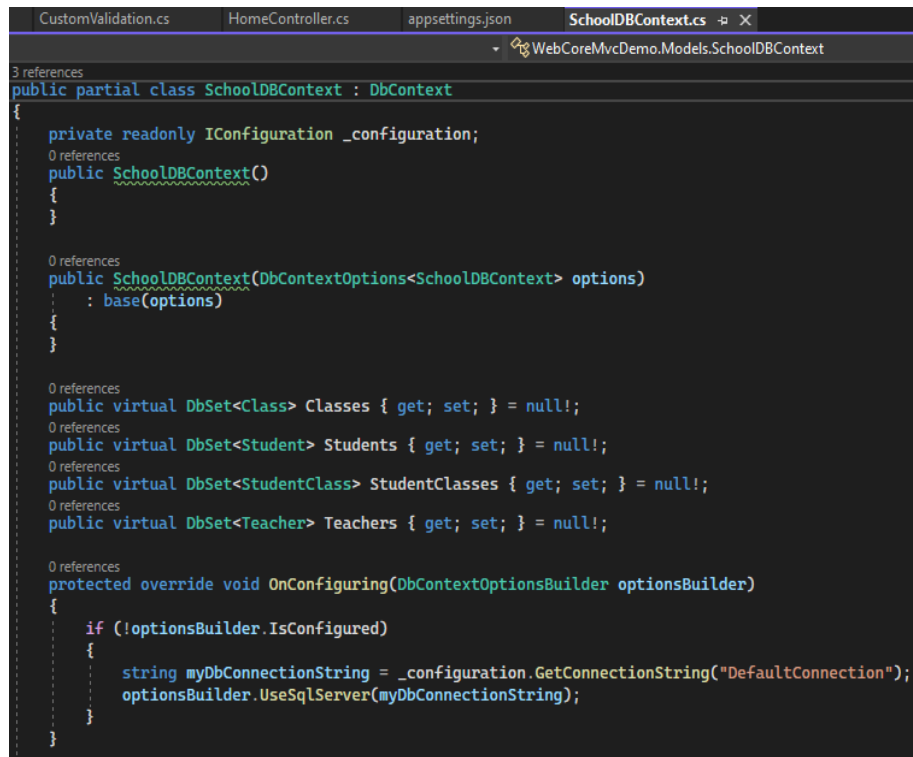


Hình 4.4.3a. CSDL bảng Student

Entity Framework Core

4.4.3. Truy xuất dữ liệu với EF Core

DB-First: PM> Scaffold-DbContext Command



```
CustomValidation.cs  HomeController.cs  appsettings.json  SchoolDbContext.cs  X
WebCoreMvcDemo.Models.SchoolDbContext

3 references
public partial class SchoolDbContext : DbContext
{
    private readonly IConfiguration _configuration;

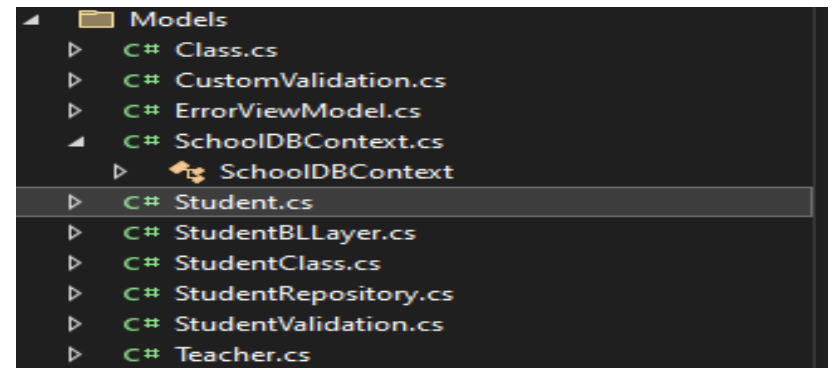
    0 references
    public SchoolDbContext()
    {
    }

    0 references
    public SchoolDbContext(DbContextOptions<SchoolDbContext> options)
        : base(options)
    {
    }

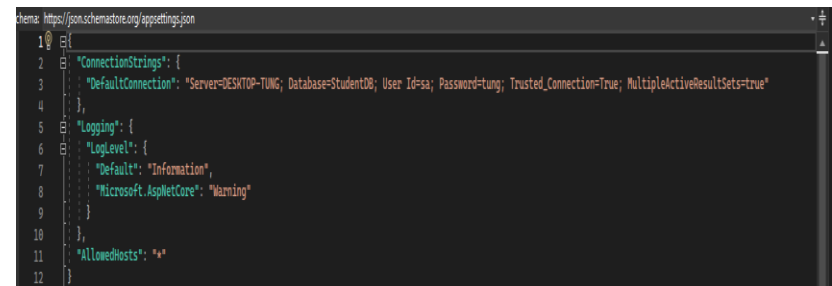
    0 references
    public virtual DbSet<Class> Classes { get; set; } = null!;
    0 references
    public virtual DbSet<Student> Students { get; set; } = null!;
    0 references
    public virtual DbSet<StudentClass> StudentClasses { get; set; } = null!;
    0 references
    public virtual DbSet<Teacher> Teachers { get; set; } = null!;

    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            string myDbConnectionString = _configuration.GetConnectionString("DefaultConnection");
            optionsBuilder.UseSqlServer(myDbConnectionString);
        }
    }
}
```

Scaffold-DbContext Command tạo DatabaseContext và Models



```
Models
├── C# Class.cs
├── C# CustomValidation.cs
├── C# ErrorViewModel.cs
├── C# SchoolDbContext.cs
│   └── SchoolDbContext
├── C# Student.cs
├── C# StudentBLLayer.cs
├── C# StudentClass.cs
├── C# StudentRepository.cs
├── C# StudentValidation.cs
└── C# Teacher.cs
```



```
Schema: https://json.schemastore.org/appsettings.json
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=DESKTOP-TUNG; Database=StudentDB; User Id=sa; Password=tung; Trusted_Connection=True; MultipleActiveResultSets=true"
4   },
5   "Logging": {
6     "LogLevel": {
7       "Default": "Information",
8       "Microsoft.AspNetCore": "Warning"
9     }
10  },
11  "AllowedHosts": "*"
12 }
```

Thiết lập ConnectionString trong appSettings.json

Entity Framework Core

4.4.3 Truy xuất CSDL với EFCore

Code-First: Tạo DbContext và DbSet

- **Tạo Model:** EF cần có mô hình dữ liệu (Entity Data Model) để giao tiếp với cơ sở dữ liệu. Nó xây dựng một mô hình dựa trên mẫu của các Domain Model và DataAnnotation (hoặc cấu hình Fluent API). EF Model bao gồm 3 phần:
 - Mô hình khái niệm: ContextClass (DomainModel và DataAnnotation)
 - Mô hình lưu trữ CSDL (Database provider và Connection String)
 - Ánh xạ giữa Concept và Storage

Ví dụ:

```
namespace EFCoreTutorials
{
    public class SchoolContext : DbContext
    {
        public DbSet<Student> Students { get; set; }
        public DbSet<Course> Courses { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=.\SQLEXPRESS;Database=SchoolDB;Trusted_Connection=true");
        }
    }
}
```

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
}

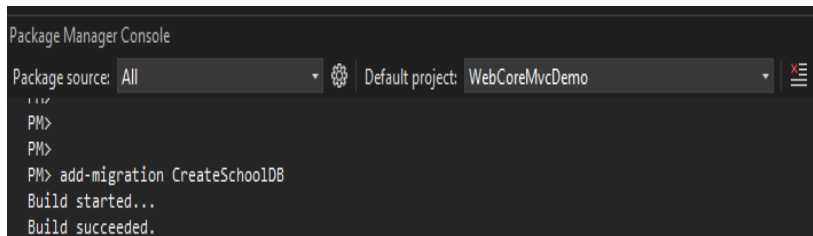
public class Course
{
    public int CourseId { get; set; }
    public string CourseName { get; set; }
}
```

Entity Framework Core

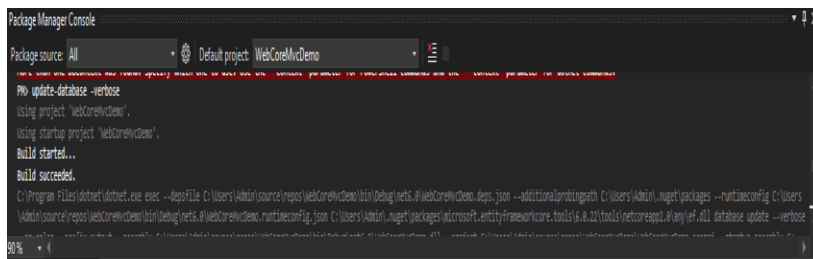
4.4.3. Truy xuất dữ liệu với EF Core

Code-First: Tạo DbContext và DbSet

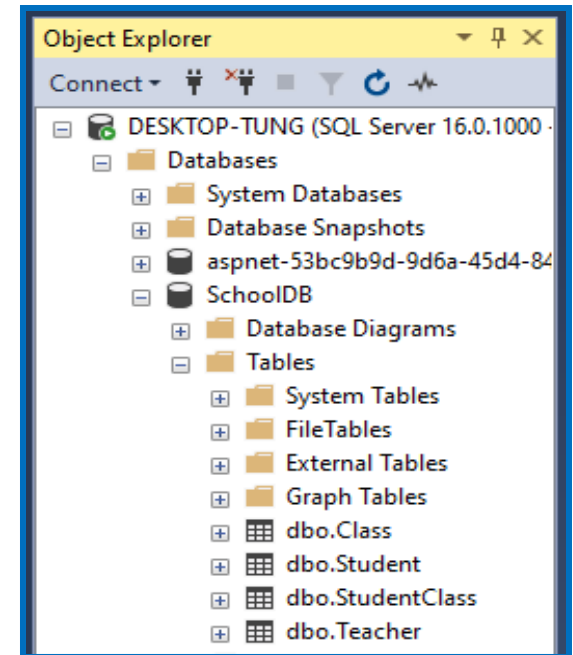
- **Migration:** EF Core sử dụng lệnh di chuyển để tạo hoặc cập nhật cơ sở dữ liệu dựa trên Models. Chúng ta có thể thực thi lệnh di chuyển bằng NuGet Package Manger Console
 - add-migration CreateDBName
 - update-database – verbose



```
Package Manager Console
Package source: All Default project: WebCoreMvcDemo
PM>
PM>
PM> add-migration CreateSchoolDB
Build started...
Build succeeded.
```



```
Package Manager Console
Package source: All Default project: WebCoreMvcDemo
PM> update-database -verbose
Using project 'WebCoreMvcDemo'.
Using startup project 'WebCoreMvcDemo'.
Build started...
Build succeeded.
C:\Program Files\dotnet\dotnet.exe exec --depsfile C:\Users\Admin\source\repos\WebCoreMvcDemo\bin\Debug\net5.0\WebCoreMvcDemo.deps.json --additionalprobingpath C:\Users\Admin\.nuget\packages --runtimeconfig C:\Users\Admin\source\repos\WebCoreMvcDemo\bin\Debug\net5.0\WebCoreMvcDemo.runtimeconfig.json C:\Users\Admin\.nuget\packages\microsoft.entityframeworkcore.tools\6.0.12\tools\netcoreapp\any\ef.dll database update --verbose
```



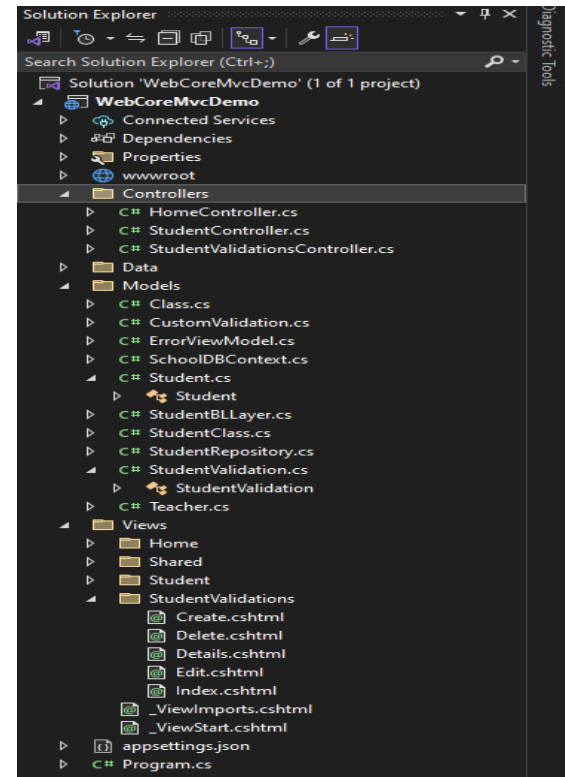
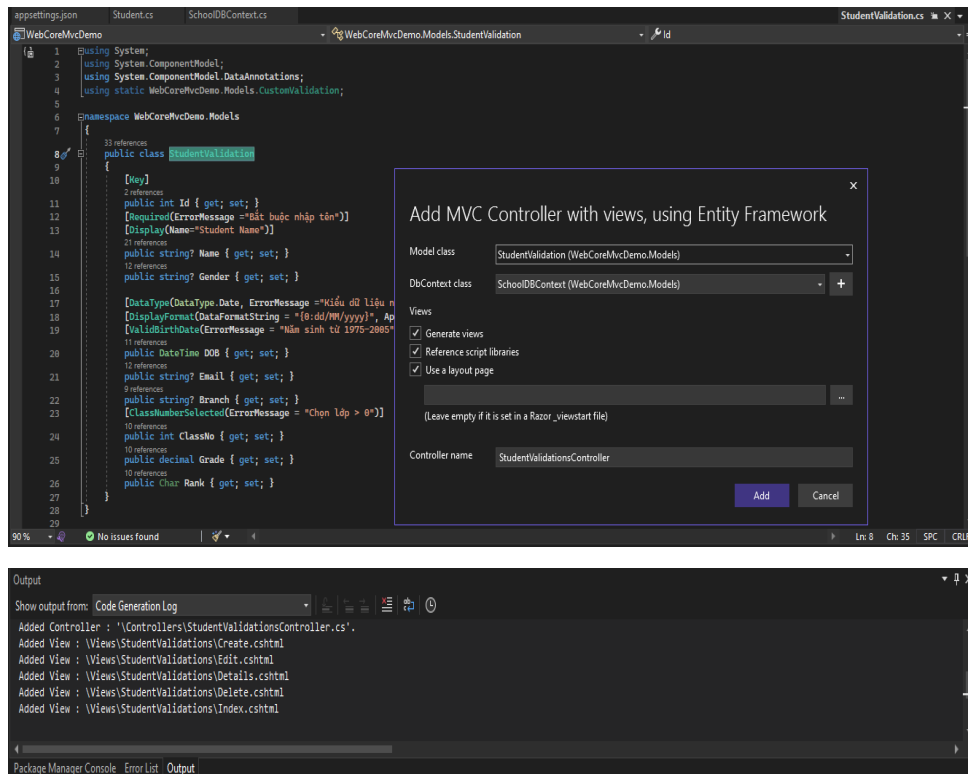
Chú ý: Nếu có cập nhật Models hoặc Config phải sync lại

Hình 4.4.3b. CSDL Migrate

Entity Framework Core

4.4.3. Truy xuất dữ liệu với EF Core

Truy xuất DB với EF Core: Tạo Controller (with View, using Entity Framework)



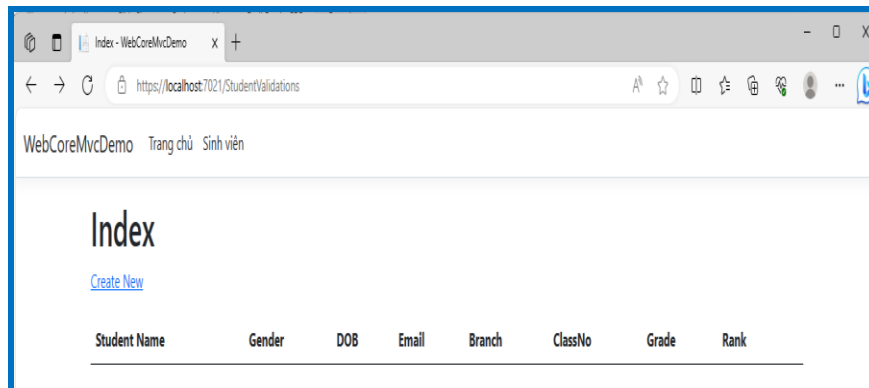
Hình 4.4.3c. Truy xuất DB với EF

Chú ý: Model cần thiết lập Key

Entity Framework Core

4.4.3. Truy xuất dữ liệu với EF Core

Truy xuất DB với EF Core: Tạo Controller (with View, using Entity Framework)



The screenshot shows a web browser window with the URL `https://localhost:7021/StudentValidations/Create`. The page title is "WebCoreMvcDemo" and the navigation bar includes "Trang chủ" and "Sinh viên". The main content area displays the title "Create StudentValidation". Below the title, there is a form with the following fields: Student Name (text input with value "Nguyễn Văn A"), Gender (text input with value "Nam"), DOB (text input with value "09/05/2000" and a calendar icon), Email (text input with value "nva@gmail.com"), Branch (text input with value "CSE"), ClassNo (text input with value "1"), Grade (text input with value "7.8"), and Rank (text input with value "B"). At the bottom of the form, there is a blue "Create" button and a link "Back to List". The footer of the page displays "© 2023 - WebCoreMvcDemo - Privacy".

KẾT THÚC

Q & A

XIN CHÀO VÀ HẸN GẶP LẠI TUẦN SAU